



Python应用基础

第6章 代码打包：函数

陈刚

中山大学岭南学院

基础篇目录

Contents

第 1 章

概述

第 2 章

Python 基本操作

第 3 章

列表和元组

第 4 章

控制结构

第 5 章

字典和集合

第 6 章

用函数打包代码

第 7 章

用文件存取数据

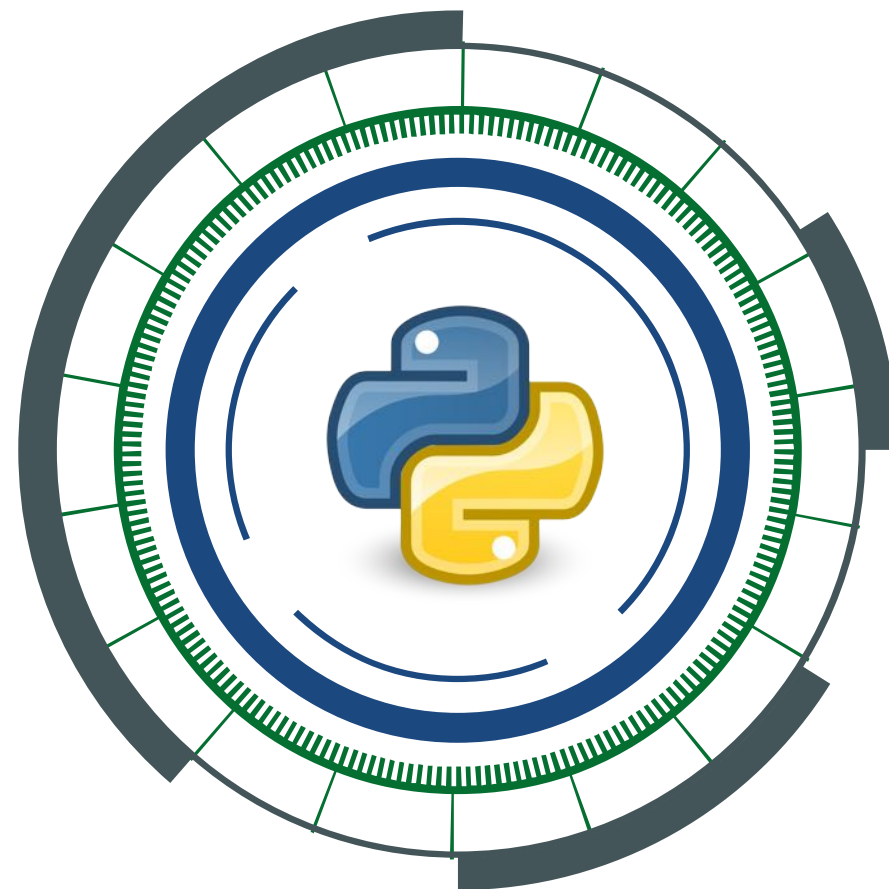


pythonTM
programming



学习目标

- 1 了解函数的作用
- 2 掌握函数定义和调用的方法
- 3 熟悉函数参数的概念和应用
- 4 熟悉局部变量和全局变量的概念及区别
- 5 掌握匿名函数的概念和应用
- 6 掌握递归函数的概念和应用
- 7 灵活运用函数解决实际问题





函数概述

函数概述

- 利用函数将一些相关语句组合在一起，从而让它们可以在程序中多次执行。
- **任务分解**：将复杂任务分解成多个相对独立的部分，对每个部分进行抽象化，得到针对某一问题通用的解决方案，然后用函数包裹这些代码。
- **代码重用**：利用函数将代码打包后，可以在多处使用。在使用过程中不再关注针对具体问题的解决方案。一次编写后，可以在多处使用，减少了代码冗余，也使得代码维护更加容易。



6.1

定义和调用函数

定义函数

- Python 中使用 `def` 语句创建函数，其一般的格式如下所示：

```
def name(arg1, arg2, ...,argN):  
    statements
```

关键字 `def` 后面的 `name` 就是函数名；

圆括号中的 `arg1`、`arg2` 和 `argN` 等是函数的参数，用于传入值，参数可以是 0 个或者多个；

缩进的代码块 `statements` 属于该函数，称为函数体。

```
❶ def repeator(s, n):  
❷     result = s * n  
❸     print(result)
```

调用函数

- 定义语句运行后会新建一个名为 repeator 的变量名，其类型为 function，即函数。

```
❶ type(repeator)  
<class 'function'>
```

- 与内置函数一样，定义并创建完函数后，可以通过函数名调用执行。

```
❷ repeator('金融科技', 3)  
金融科技金融科技金融科技
```



```
# 先调用
say_hello() # NameError: name 'say_hello' is not defined
```

```
# 后定义
def say_hello():
    print("Hello!")
```

```
# 先定义
def say_hello():
    print("Hello!")
```

```
# 后调用
say_hello() # 输出: Hello!
```

- 在Python中，不可以直接先调用函数，然后再定义它。
- Python是**解释型语言**，代码从上到下逐行执行。当执行到函数调用时，Python必须已经知道该函数的存在。

定义函数返回值

- 在很多情况下，函数需要将计算结果返回到调用处。在这类函数的函数体中，通常包含一条 `return` 语句：

```
def name(arg1, arg2, ...,argN):  
    statements  
    return value
```

`return` 语句不一定出现在函数体的最后，而是可以在任何位置。

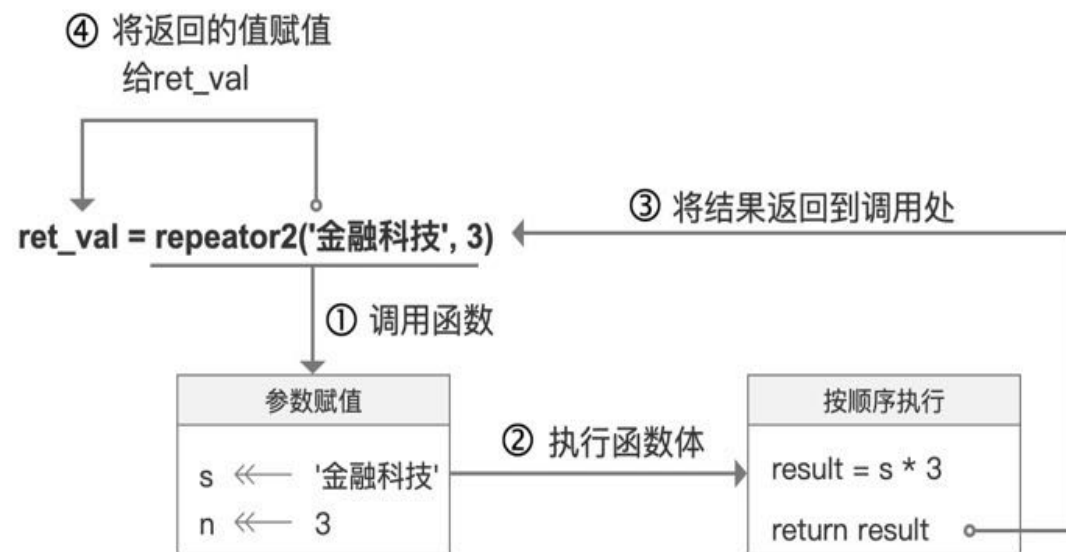
只要执行到 `return` 语句，函数就结束，将 `value` 返回到调用处。

实际上，即使在创建函数时，没有在函数体中添加 `return` 语句，Python 也会默默地在函数体最后添加一条 `return None`。

定义函数返回值

- 下面的代码对 repeator 函数进行改造，对重复后的字符串不在函数体中打印，而是返回到调用处，由调用者来处理：

```
❶ def repeator2(s, n):  
❷     result = s * n  
❸     return result  
  
❹ ret_val = repeator2('金融科技', 3)  
❺ ret_val  
  
'金融科技金融科技金融科技'
```



定义函数返回值

- 在 Python 中，还允许在函数中返回多个值。只需将返回值以逗号隔开，放在 `return` 关键字后面即可：

```
❶ def calculator(m, n):  
❷     return m + n, m - n, m * n, m / n  
❸ i, j = 10, 8  
❹ result1, result2, result3, result4 = calculator(i, j)  
❺ print('{} 和 {} 的加减乘除运算结果是: {}, {}, {}, {}'.format(i, j,  
                                                                    result1, result2, result3, result4))
```

10 和 8 的加减乘除运算结果是: 18, 2, 80, 1.25

调用函数的四个步骤

- ① 程序执行到函数调用时，在调用处暂停，等待函数执行完毕；
- ② 将实参赋值给函数的形参；
- ③ 执行函数体中的语句；
- ④ 调用结束后，回到调用前暂停处继续执行，如果函数体中执行了 `return` 语句，`return` 关键字后的值会返回到暂停处，供程序使用，否则函数返回 `None` 值。

练习1：制作“专属咖啡”

练习内容：

- 创建一个名为 `make_coffee(size, kind)` 的函数。
- 参数 `size` 代表大小，`kind` 代表咖啡种类。
- 函数不需要返回值，只需在内部打印一句话，例如：

您点了一杯“超大杯”杯的“焦糖玛奇朵”。正在为您制作，请稍等...



案例：个人所得税计算器

案例：个人所得税计算器

- 所谓个人所得税（personal income tax），是以个人取得的各项应税所得为对象征收的一种税，是调整征税机关与自然人（居民、非居民个人）之间在个人所得税的征纳与管理过程中所发生社会关系的法律规范的总称。

个人综合所得适用 7 级超额累进税率，即把征税对象的数额划分为若干等级，对每个等级部分的数额分别规定相应税率，分别计算税额，各级税额之和为应纳税额。也就是说征税对象数额超过某一等级时，仅就超过部分，按高一级的税率计算征税额。因此，通常应交所得税计算公式为：

$$\text{应交所得税} = \text{应税所得} * \text{适用税率} - \text{速算扣除数}$$

案例：个人所得税计算器

- 公式中的速算扣除数是按全额累进方法计算出的税额比按超额累进方法计算出的税额多，即有重复计算的部分，如表所示：

综合所得 7 级超额累进税率及速算扣除表

级数	全年应纳税所得额	适用税率	速算扣除数
1	不超过 36 000 元的部分	3%	0
2	超过 36 000 至 144 000 元的部分	10%	2 520
3	超过 144 000 至 300 000 元的部分	20%	16 920
4	超过 300 000 至 420 000 元的部分	25%	31 920
5	超过 420 000 至 660 000 元的部分	30%	52 920
6	超过 660 000 至 960 000 元的部分	35%	85 920
7	超过 960 000 的部分	45%	181 920

【例】编写程序，实现个人所得税计算器。用户输入全年综合所得以及各项扣除之和后，计算出应纳税额和税后收入。

案例：个人所得税计算器

主程序

1. 接受用户输入全年综合所得year_income以及各项扣除之和reduce_amt
2. 调用getTaxAmt, 获得应纳税额
3. 打印结果

year_income
reduce_amt

①

tax

④

getTaxAmt

1. 根据参数year_income和reduce_amt计算应纳税所得tax_income
2. 调用getTaxRateAndDeduction函数, 获得税率tax_rate和速算扣除数deduction
3. 计算并返回应纳税额tax

tax_income

②

tax_rate
deduction

③

getTaxRateAndDeduction

1. 根据参数tax_income, 获得税率tax_rate和速算扣除deduction
2. 返回tax_rate和deduction

案例：个人所得税计算器

```
1 def getTaxRateAndDeduction(tax_income):
2     if tax_income <= 36000:
3         tax_rate, deduction = 0.03, 0
4     elif tax_income <= 144000:
5         tax_rate, deduction = 0.1, 2520
6     elif tax_income <= 300000:
7         tax_rate, deduction = 0.2, 16920
8     elif tax_income <= 420000:
9         tax_rate, deduction = 0.25, 31920
10    elif tax_income <= 660000:
11        tax_rate, deduction = 0.30, 52920
12    elif tax_income <= 960000:
13        tax_rate, deduction = 0.35, 85920
14    else:
15        tax_rate, deduction = 0.4, 181920
16    return tax_rate, deduction
17
18 def getTaxAmt(year_income, reduce_amt):
19     tax_income = year_income - reduce_amt - 60000
20     if tax_income < 0:
21         return 0
22     else:
23         rate, deduction_amt = getTaxRateAndDeduction(tax_income)
24         tax = tax_income * rate - deduction_amt
25     return tax
26
27 user_year_income = float(input(' 请输入全年综合所得: '))
28 print(' 各项扣除之和是指专项扣除、专项附加扣除和依法确定的其他扣除之和。')
29 user_reduce_amt = float(input(' 请输入全年各项扣除之和: '))
30 user_tax = getTaxAmt(user_year_income, user_reduce_amt)
31 print(' 全年应纳税额为: {:.2f} 元 \n 全年税后收入为: {:.2f} 元 '.format(
32     user_tax, user_year_income - user_tax))
```

请输入全年综合所得: 1000000
各项扣除之和是指专项扣除、专项附加扣除和依法确定的其他扣除之和。
请输入全年各项扣除之和: 24000
全年应纳税额为: 234680.00 元
全年税后收入为: 765320.00 元



函数参数

参数传递

- 参数的传递过程，实际上是一个赋值的过程。在调用函数时，调用者的实际参数自动赋值给函数的形式参数变量。

```
❶ def avg(m, n):  
❷     return (m + n) / 2  
❸ avg(10, 8)  
9.0
```

不可变和可变类型参数

● 在函数中使用时，可变和不可变类型参数的表现有所不同。

● 下面是一个以不可变类型浮点型为参数例子：

```
❶ def priceChanger(p):  
❷     p = p + 10  
❸     print('改变后的价格: {:.2f}'.format(p))  
❹ price = 10.8  
❺ priceChanger(price)  
    改变后的价格: 20.80  
❻ print(price)  
    10.8
```

不可变和可变类型参数

● 下面是一个以可变类型为参数例子：

```
❶ def contentChanger(name_list):  
❷     name_list[0], name_list[1] = name_list[1], name_list[0]  
❸     print('函数中的 name_list:', name_list)  
❹ bank_name = ['农业银行', '建设银行']  
❺ contentChanger(bank_name)  
❻ print(' 调用函数后的 bak_name:', bank_name)
```

函数中的 name_list: ['建设银行', '农业银行']
调用函数后的 bak_name: ['建设银行', '农业银行']

可以使用列表 `copy` 方法或者使用分片操作创建新列表，以防止修改：

```
❷ bank_name = ['农业银行', '建设银行']  
❸ contentChanger(bank_name.copy())  
❹ print('调用函数后的 bak_name:', bank_name)
```

函数中的 name_list: ['建设银行', '农业银行']
调用函数后的 bak_name: ['农业银行', '建设银行']

位置参数调用

- 位置参数是调用函数为形参赋值的一种默认方式。实参与形参按照从左到右的位置顺序依次赋值：

```
❶ def myMinus(num1, num2):  
❷     return num1 - num2  
❸ myMinus(10, 8)  
2
```

- 赋值顺序改变将得到不同的结果：

```
❹ myMinus(8, 10)  
-2
```


关键字参数调用

- 为了避免位置参数赋值带来的混乱，Python 允许调用函数时通过关键字参数的形式指定形参与实参的对应关系。调用者使用 `name=value` 的形式来指定函数中的哪个形参接受某个值。关键字参数起到了数据标签的作用。

```
❶ print(myMinus(num1=10, num2=8))  
❷ print(myMinus(num2=8, num1=10))  
2  
2
```

位置参数和关键字参数是调用函数时，对形参赋值的两种方式，与函数定义无关。

指定默认参数值

- 在函数定义时，可以为参数指定值。当调用者没有提供对应参数值时，就可以使用指定的默认值。指定默认参数值在 Python 的函数中广泛存在。例如，打印函数 `print`，在查看其帮助时，其函数的部分描述如下：

❶ `help(print)`

Help on built-in function print in module builtins:

`print(...)`

`print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)`

Prints the values to a stream, or to `sys.stdout` by default. Optional keyword arguments:

`file`: a file-like object (stream); defaults to the current `sys.stdout`.

`sep`: string inserted between values, default a space.

`end`: string appended after the last value, default a newline.

`flush`: whether to forcibly flush the stream.

❷ `print('中国银行', '农业银行', '工商银行', '建设银行')`

❸ `print('中国银行', '农业银行', '工商银行', '建设银行', sep='_')`

中国银行 农业银行 工商银行 建设银行

中国银行_农业银行_工商银行_建设银行

指定默认参数值

- 在定义函数时，为形参指定默认值，就可以让该形参在调用时变为可选。

```
❶ def myMod(x, y=2):  
❷     return x % y  
❸ print(myMod(15,4))  
❹ print(myMod(15))
```

3

1

任意数量位置参数

- 使用单星号“*”收集位置参数。单个星号将一组可变数量的位置参数组合成参数值的元组。在函数内部可以通过访问元组中的每个元素来使用参数。

```
① def priceAvg(*prices):  
②     total_price = 0  
③     for p in prices:  
④         total_price += p  
⑤     return total_price / len(prices)  
  
⑥ priceAvg(10.0, 20.0, 30.0)  
20.0
```

任意数量参数

【例】 在一些竞赛中，为了取得一个相对公平的分數，通常选手的最终得分是去掉一个最高分和一个最低分后的平均分。编写函数，输入选手的姓名以及各个评委给出的分數，打印出选手姓名和最终得分。

```
❶ def finalScore(name, *scores):  
    if len(scores) < 3:  
        print('{} 的评分不足 3 个.'.format(name))  
    else:  
        max_score = max(scores)  
        min_score = min(scores)  
        score_list = list(scores)  
        score_list.remove(max_score)  
        score_list.remove(min_score)  
  
        print('{} 的最终得分为 {:.1f} 分.'.format(name,  
                                                    sum(score_list)/len(score_list)))  
❷ finalScore('宋俊', 10, 8.5)  
❸ finalScore('张宁', 9.9, 10, 8.5, 9.0, 9.2, 10)
```

宋俊的评分不足 3 个。
张宁的最终得分为 :9.5 分。

任意数量关键字参数

- 针对形参的关键字参数赋值形式，利用 Python 定义函数时，在形参前面加上双星号“**”来定义收集关键字参数的形参。此时形参是字典类型。当调用函数时，参数赋值使用 `name=value` 的形式时，`name` 是形参字典的一个键，`value` 是该键对应的值。

结合字典方法，这种关键字参数收集器可以轻松实现更加灵活的函数功能。

任意数量关键字参数

【例】编写函数，实现简单的人事信息展示功能。注意：除了姓名外，其他信息可能不完整。

```
❶ def empDetailInfo(**emp_info):
❷     if 'name' not in emp_info.keys():
        print('必须有姓名信息。')
    else:
        print(emp_info['name'] + ' 的信息如下：')
❸         print('部门：' + emp_info.get('department', ''))
❹         print('电话号码：' + emp_info.get('phone', ''))
❺         print('电子邮箱：' + emp_info.get('email', ''))

❻ empDetailInfo(name='张珊', department='管理', email='zhangs@163.com')
        print('-' * 30)
❼ empDetailInfo(name='赵柳', department='财务', phone='186123456789')
```

张珊的信息如下：

部门：管理

电话号码：

电子邮箱：zhangs@163.com

赵柳的信息如下：

部门：财务

电话号码：186123456789

电子邮箱：

解包参数

- 在调用函数时，实参也可以使用“*”和“**”语法。此时不是收集参数，正好相反，实参前加上“*”或“**”执行的是参数解包。
- 通常来说，在列表、元组等类型的实参值前加上“*”，将这些类型的元素解包成位置参数的形式；在字典类型的实参值前加上“**”，将字典的元组解包成关键字参数的形式。

解包参数

【例】对人事信息展示例子中的函数定义进行以下改造：

```
❶ def empDetailInfo(name, department='', phone='', email=''):
❷     if name and len(name) > 0:
❸         print(name + ' 的信息如下: ')
❹         print('部门: ' + department)
❺         print('电话号码: ' + phone)
❻         print('电子邮箱: ' + email)
❼     else:
❽         print('必须有姓名信息。')
```

当调用者的数据存储在列表中时，通过在前面加上“*”对列表解包来实现位置参数形式的调用：

```
❸ emp_info1 = ['张明', '管理', '18612345683', 'zhangming@qq.com']
❹ empDetailInfo(*emp_info1)
❺ empDetailInfo('张明', '管理', '18612345683', 'zhangming@qq.com')
```

```
张明 的信息如下:
部门: 管理
电话号码: 18612345683
电子邮箱: zhangming@qq.com
张明 的信息如下:
部门: 管理
电话号码: 18612345683
电子邮箱: zhangming@qq.com
```

解包参数

【例续】当调用者的数据存储在字典中时，可以通过在字典前加上“**”对字典解包来实现关键字参数形式的调用：

```
❶ emp_info2 = {'name': '张珊', 'department': '管理', 'email': 'zhangs@163.com'}  
❷ empDetailInfo(**emp_info2)  
❸ empDetailInfo(name='张珊', department='管理', email='zhangs@163.com')
```

张珊 的信息如下：

部门：管理

电话号码：

电子邮箱：zhangs@163.com

张珊 的信息如下：

部门：管理

电话号码：

电子邮箱：zhangs@163.com

练习2： 反转字符串

练习内容：

- 请编写一个名为 `any_join` 的函数。
- 它可以使用 `*args` 接收任意数量的字符串参数。
- 再接收一个名为 `separator` 的关键字参数，默认值为 `'-'`。
- 函数功能是：使用 `separator` 将所有传入的字符串连接起来，并打印结果。
- 示例调用：
`any_join("苹果", "香蕉", "橘子", separator=" + ")` 应输出：苹果 + 香蕉 + 橘子



变量作用域

变量作用域概述

- Python 中规定每个变量都有它的作用域，即变量只有在作用域范围内才是可见可用的。
- 作用域能避免程序代码中的名称冲突，在一个函数中定义的变量名称不会干扰另外一个函数内的变量。这有助于使函数更加独立。
- 在赋值时变量就和某一个特定的作用域绑定了。根据作用域范围的大小，可以将作用域分为全局作用域和局部作用域。

局部变量

- 局部变量仅仅在局部作用域内可用。在局部作用域之外，该变量不可见。
- 如果变量是在函数体内被创建（首次赋值）的，这个变量就只能在函数体内使用，是这个函数体的局部变量。函数执行结束后，局部变量被销毁。

```
❶ def avg(x, y):  
❷     avg_price = (x + y) / 2  
❸     print(avg_price)
```

```
❹ avg(10.0, 20.0)
```

15.0

```
❺ print(avg_price)
```

Traceback (most recent call last):
File "<pyshell#5>", line 1, in <module>
 print(avg_price)
NameError: name 'avg_price' is not defined

```
def avg(x, y):
```

```
    avg_price = (x + y) / 2
```

```
    print(avg_price)
```

```
avg(10.0, 20.0)
```

全局变量

- 全局作用域是相对于局部作用域来说的概念。
- 在 Python 中，全局作用域的作用范围是单个文件。相对于局部变量，在所有函数之外创建的变量，在整个文件范围内都是可见的，即全局变量。

```
❶ rate = 0.03
❷ def taxCalculator(tax_income):
❸     return tax_income * rate

❹ tax = taxCalculator(20000)
❺ print('利用税率 {:.2%} 计算的应纳税额为 {:.2f} 元 '.format(rate, tax))
```

利用税率 3.00% 计算的应纳税额为 :600.00 元

通过全局变量可以在函数之间进行通信，也就是共享信息。但全局变量破坏了函数的独立性，因此除非有充分的理由，否则函数应该尽量地依赖形参和返回值，而不是全局变量。

同名的局部变量和全局变量

- 在同名的情况下，在局部作用域内，可见的是局部变量，全局变量被暂时隐藏起来。

```
1 rate = 0.03 # 全局变量 rate
2 def taxCalculator2(tax_income):
3     if tax_income < 3000:
4         rate = 0.03 # 局部变量 rate
5     else:
6         rate = 0.1 # 局部变量 rate
7     print('利用税率 {:.2%} 计算的应纳税额为 :{:.2f}元'.format(rate, tax_income * rate))
8
9 taxCalculator2(20000)
print('调用函数后的税率是: {:.2%}'.format(rate))
```

利用税率 10.00% 计算的应纳税额为 :2000.00元
调用函数后的税率是: 3.00%

global 语句

- 关键字 global 语句通常放在函数体的开始部分，其语法如下：

```
global variable_names
```

在 global 后面是需要声明为全局变量的变量列表。

```
① rate = 0.03 # 全局变量 rate
② def taxCalculator2(tax_income):
③     global rate
④     if tax_income < 3000:
⑤         rate = 0.03 # 局部变量 rate
⑥     else:
⑦         rate = 0.1 # 局部变量 rate
⑧     print('利用税率 {:.2%} 计算的应纳税额为 :{:.2f}元'.format(rate, tax_income * rate))

⑨ taxCalculator2(20000)
⑩ print('调用函数后的税率是:{:.2f} '.format(rate))
```

利用税率 10.00% 计算的应纳税额为 :2000.00元
调用函数后的税率是: 10.00%

练习3：作用域的力量

练习内容：

- 以下代码，不运行
- 分组讨论，预测输出结果。

```
treasure = 100
def hunt():
    treasure = 50
    print("函数内：", treasure)
hunt()
print("全局：", treasure)
def real_hunt():
    global treasure
    treasure = 200
real_hunt()
print("修改后：", treasure)
```



匿名函数 LAMBDA

匿名函数定义

- Python 中提供了一项非常有用的功能：利用 lambda 函数来替代 def，创建一个临时简单函数。
- 与 def 语句不同，lambda 是一个表达式。
- 定义匿名函数的语法如下所示：

```
lambda <args>: <expression>
```

匿名函数的函数体只有一个表达式，类似于普通函数 return 后面的表达式。

```
❶ getSecondItem = lambda x: x[1]  
❷ print(type(getSecondItem))  
❸ print(getSecondItem(['600000', '浦发银行']))
```

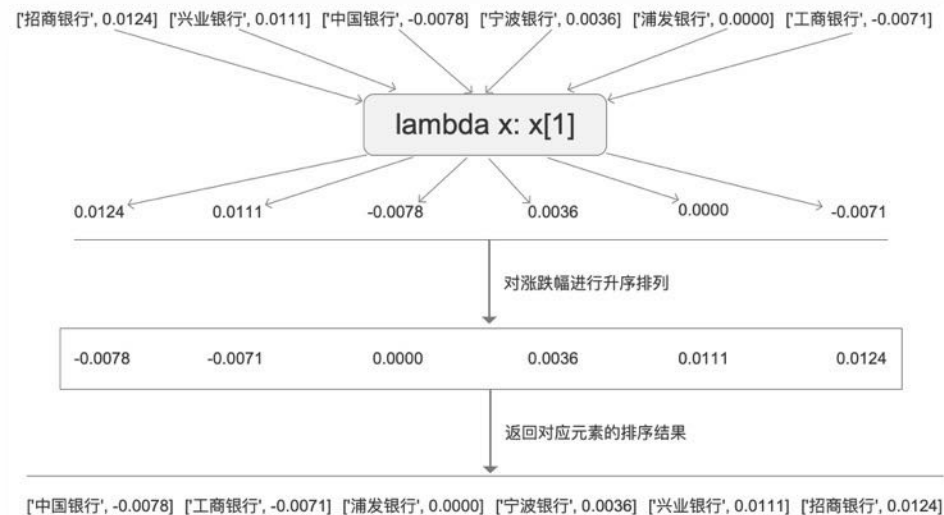
```
<class 'function'>  
浦发银行
```

匿名函数应用一：列表排序

【例】 有二维列表存储了股票的名称和涨跌幅。编写程序，将列表按照涨跌幅的升序排列。

```
❶ stock_list = [['招商银行', 0.0124],  
                 ['兴业银行', 0.0111],  
                 ['中国银行', -0.0078],  
                 ['宁波银行', 0.0036],  
                 ['浦发银行', 0.0000],  
                 ['工商银行', -0.0071]]  
❷ stock_list.sort(key=lambda x: x[1])  
❸ for name, pct in stock_list:  
    print(name, pct)
```

```
中国银行 -0.0078  
工商银行 -0.0071  
浦发银行 0.0  
宁波银行 0.0036  
兴业银行 0.0111  
招商银行 0.0124
```



练习4：极速排序

练习内容：

- 有一个列表，里面装着学生信息，格式为（姓名，年龄，成绩）。

```
students = [("张三", 20, 88), ("李四", 19, 95), ("王五", 21, 85)]
```

- 请使用 `sorted()` 函数和 `lambda` 表达式，按成绩从高到低对学生进行排序。
- 请使用 `sorted()` 函数和 `lambda` 表达式，按年龄从小到大对学生进行排序。
- 如果按姓名长度排序，`lambda` 表达式应该怎么写？

匿名函数应用二：映射函数 map

【例】 当收集股票信息时，无论是从文件读取，还是从网络获取，元素通常来说都是字符串类型。但是对于涨跌幅来说，应该是浮点型数据。因此，常常需要对每个涨跌幅进行数据转换。

```
❶ stock_list = [['招商银行', '0.0124'], ['兴业银行', '0.0111'],  
                ['中国银行', '-0.0078'], ['宁波银行', '0.0036'],  
                ['浦发银行', '0.0000'], ['工商银行', '-0.0071']]
```

```
❷ stock_map = map(lambda x: [x[0], float(x[1])], stock_list)  
print(type(stock_map))
```

```
<class 'map'>
```

```
❸ stock_list = list(stock_map)  
print(stock_list)
```

```
[['招商银行', 0.0124], ['兴业银行', 0.0111], ['中国银行', -0.0078], ['宁波银行', 0.0036],  
 ['浦发银行', 0.0], ['工商银行', -0.0071]]
```

匿名函数应用三：选择函数 filter

【例】 利用 filter 函数筛选出列表中上涨的股票信息。

```
❶ stock_list = [['招商银行', 0.0124], ['兴业银行', 0.0111],  
                ['中国银行', -0.0078], ['宁波银行', 0.0036],  
                ['浦发银行', 0.0000], ['工商银行', -0.0071]]
```

```
❷ stock_filter = filter(lambda x: x[1]>0, stock_list)  
   print(type(stock_filter))
```

```
<class 'filter'>
```

```
❸ stock_list = list(stock_filter)  
   print(stock_list)
```

```
[['招商银行', 0.0124], ['兴业银行', 0.0111], ['宁波银行', 0.0036]]
```


练习5：英雄攻击力计算器

练习内容：

- 有一个英雄的攻击力列表： `attacks = [100, 250, 300, 450, 80, 600, 30]`
 - 使用 `filter` 和一个 `lambda` 函数，筛选出攻击力 大于 200 的英雄
 - 使用 `map` 和一个 `lambda` 函数，将所有筛选后的攻击力 提升 10%（乘以1.1）
- 挑战： 将上述两步用一行代码链式完成。



递归函数

递归函数概述

- 递归是一种广泛应用算法。它能够把一个大型复杂的问题转化为一个与原问题相似的较小规模的问题来求解，用非常简洁的方法来解决重要问题。
- 阶乘是数学上经典的递归例子。一个正整数的阶乘（factorial）是所有小于等于该数的正整数的乘积，特别地，0 的阶乘为 1。自然数 n 的阶乘表示为 $n!$ 。根据定义可知：

$$n! = \begin{cases} 1 & n = 0 \\ n * (n - 1)! & n > 0 \end{cases}$$

递归函数概述

- 对于递归来说，必须存在一个或多个基例，基例不再需要递归，是一个确定的值或表达式。所有递归链都要以一个或多个基例结束。
- 递归函数的核心是函数定义中调用了函数自身。

递归函数案例：阶乘计算

【例】根据用户输入的整数 n ，计算并输出 n 的阶乘值。

```
① def factorial(n):  
②     if n == 0:  
③         return 1  
④     else:  
⑤         return n * factorial(n-1)  
  
⑥ num = int(input("请输入一个正整数 : "))  
⑦ print('{} 的阶乘为:{}'.format(num, factorial(num)))
```

请输入一个整数 : 5

5 的阶乘为 :120

递归函数案例：斐波那契数列

【例】斐波那契数列（Fibonacci sequence），又称黄金分割数列。因数学家列昂纳多·斐波那契（Leonardoda Fibonacci）以兔子繁殖为例子而引入，故又称为“兔子数列”，指的是这样一个数列：1、1、2、3、5、8、13、21、34……
编写程序，用户输入正整数 n，输出斐波那契数列的前 n 项。

```
① def fibo(i):  
②     if i in (1,2):  
③         return 1  
④     else:  
⑤         return fibo(i-1) + fibo(i-2)  
  
⑥ num = int(input('请输入一个大于 3 的正整数:'))  
⑦ print('\n斐波那契数列的前 {} 项为: '.format(num))  
⑧ for i in range(1, num+1):  
    print(fibo(i), end=' ')
```

请输入一个正整数:10

斐波那契数列的前 10 项为: 1 1 2 3 5 8 13 21 34 55

递归函数案例：斐波那契数列

【例续】前面代码在 n 的值较大时运算非常慢。通过分析可以发现，利用简单递归方式来实现时，有许多重复运算的过程。如果改变一下算法，将已经计算过的值保存下来，在需要时直接使用，这样效率会更高。

```
① calculate_dic = {1: 1, 2: 1}
② def fibByDic(n):
③     if n not in calculate_dic:
④         new_value = fibByDic(n-1) + fibByDic(n-2)
⑤         calculate_dic[n] = new_value
⑥     return calculate_dic[n]

⑦ num = int(input('请输入一个大于 3 的正整数 :'))
⑧ print('\n斐波那契数列的前 {} 项为: '.format(num))
⑨ for i in range(1, num + 1):
    print(fibByDic(i), end=' ')
```

请输入一个大于 3 的正整数 :20

斐波那契数列的前 20 项为： 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610
987 1597 2584 4181 6765

练习6：递归的乐趣——数洋葱

练习内容：

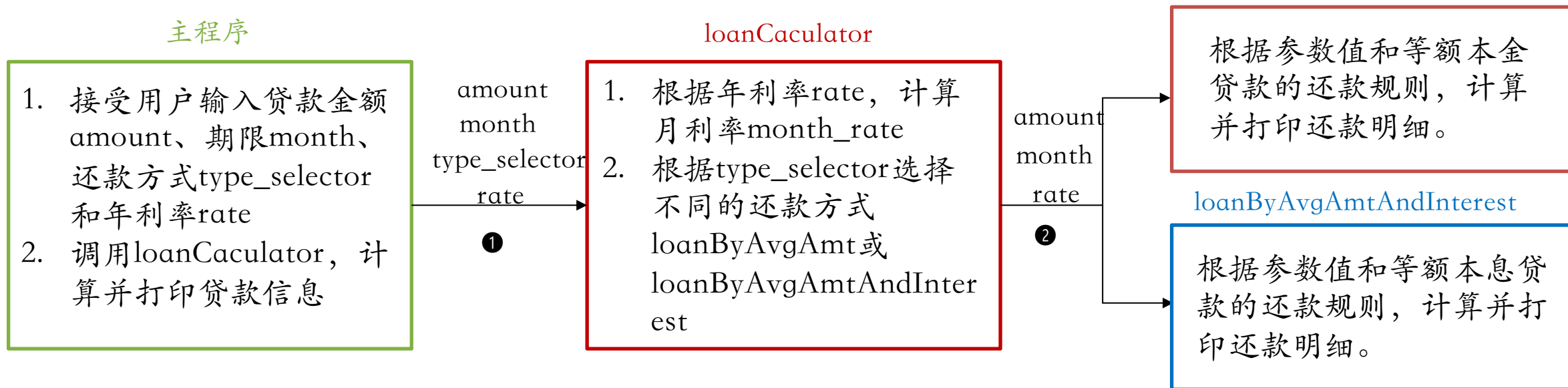
- 想象一个“俄罗斯套娃”或“洋葱”。编写一个递归函数 `peel_onion(layer)`。
- 它接收一个参数 `layer`，表示当前是第几层。
- 如果 `layer` 大于 0，函数打印：“正在拨开第 {layer} 层... 🍅”，然后调用自身，层数减 1。
- 如果 `layer` 等于 0，函数打印：“哇！拨完了，什么都没有，我被辣哭了！😭”，并停止（这就是基例）。
- 最后，调用 `peel_onion(5)` 看看效果。



案例：个人贷款计算器

案例：个人贷款计算器

【例】 个人贷款的还款方式分为等额本息还款和等额本金还款两种方式。在前面案例中，分别实现了两种还款方式。利用函数，可以实现更加复杂的系统。现要求编写个人贷款计算器，根据用户输入的贷款金额、贷款期限、还款方式和贷款年利率，打印出支付总利息和每个月的详细信息（包含还款本金、利息和剩余金额），其中贷款年利率默认为 4.9%。



案例：个人贷款计算器

等额本金还款方式函数

```
① # 等额本金贷款
def loanByAvgAmt(loan_amt, loan_month, month_rate):
    repaid_amt = 0
    #每月还款的等额本金
    loan_amt_per_month = loan_amt / loan_month
    ③ need_paid_interest = 0
    ④ for i in range(1, loan_month + 1):
        #每月还款的利息
        ⑤ interest_amt = (loan_amt - repaid_amt) * month_rate
        #所有利息的和
        ⑥ need_paid_interest = need_paid_interest + interest_amt
        #每月还款的总额：还款本金 + 当月利息
        ⑦ amt_for_month = loan_amt_per_month + interest_amt
        #已还款的本金金额
        ⑧ repaid_amt = repaid_amt + loan_amt_per_month
        ⑨ print('第{}个月需还:{:.2f},其中本金 {:.2f},利息 {:.2f}。本金余额:{:.2f}'.format(i, amt_for_month,
                                                                 loan_amt_per_month, interest_amt, loan_amt - repaid_amt))
    ⑩ print('等额本金贷款支付的利息总额为: {:.2f}元'.format(need_paid_interest))
```

案例：个人贷款计算器

等额本息还款方式函数

```
# 等额本息贷款
def loanByAvgAmtAndInterest(loan_amt, loan_month, month_rate):
    repaid_amt = 0
    #每月还款的等额本息计算公式
    need_paid_per_month = loan_amt * ((month_rate * (1 + month_rate) ** loan_month) /
                                       ((1 + month_rate) ** loan_month - 1))

    need_paid_interest = 0
    for i in range(1, loan_month + 1):
        #每月还款的利息
        interest_amt = (loan_amt - repaid_amt) * month_rate
        #所有利息的和
        need_paid_interest = need_paid_interest + interest_amt
        #每月还款的等额本息中的本金
        repaid_amt_for_month = need_paid_per_month - interest_amt
        #已还款的本金金额
        repaid_amt = repaid_amt + repaid_amt_for_month
        print('第{}个月需还:{:.2f},其中本金:{:.2f},利息:{:.2f}。本金余额:{:.2f}'.format(i,
                                              need_paid_per_month, repaid_amt_for_month, interest_amt, loan_amt - repaid_amt))
    print('等额本息贷款支付的利息总额为: {:.2f}元'.format(need_paid_interest))
```

案例：个人贷款计算器

个人贷款计算器函数及主函数

```
def loanCaculator(loan_amt, loan_month, loan_type , year_rate = 0.049):  
    ① month_rate = year_rate / 12  
    ② if loan_type == 1:  
        loanByAvgAmt(loan_amt, loan_month, month_rate)  
    elif loan_type == 2:  
        loanByAvgAmtAndInterest(loan_amt, loan_month, month_rate)  
    else:  
        print('贷款方式输入错误。')  
  
    ③ amount = float(input('请输入贷款金额: '))  
    ④ month = int(input('请输入贷款期限(月): '))  
    ⑤ type_selector = int(input('请输入还款方式(1为等额本金, 2为等额本息): '))  
    ⑥ rate_str = input('请输入贷款年利率(直接回车为默认利率4.9%): ')  
    ⑦ if rate_str and len(rate_str) > 0:  
        ⑧ loanCaculator(amount, month, type_selector, float(rate_str))  
    ⑨ else:  
        ⑩ loanCaculator(amount, month, type_selector)
```

案例：个人贷款计算器

运行并输入数据

请输入贷款金额：1000000
请输入贷款期限（月）：360
请输入还款方式（1为等额本金，2为等额本息）：1
请输入贷款年利率（直接回车为默认利率4.9%）：

第1个月需还:6861.11,其中本金2777.78,利息4083.33。本金余额:997222.22
第2个月需还:6849.77,其中本金2777.78,利息4071.99。本金余额:994444.44
第3个月需还:6838.43,其中本金2777.78,利息4060.65。本金余额:991666.67
第4个月需还:6827.08,其中本金2777.78,利息4049.31。本金余额:988888.89
第5个月需还:6815.74,其中本金2777.78,利息4037.96。本金余额:986111.11

.....

第356个月需还:2834.49,其中本金2777.78,利息56.71。本金余额:11111.11
第357个月需还:2823.15,其中本金2777.78,利息45.37。本金余额:8333.33
第358个月需还:2811.81,其中本金2777.78,利息34.03。本金余额:5555.56
第359个月需还:2800.46,其中本金2777.78,利息22.69。本金余额:2777.78
第360个月需还:2789.12,其中本金2777.78,利息11.34。本金余额:0.00

等额本金贷款支付的利息总额为：737041.67元

请输入贷款金额：1000000
请输入贷款期限（月）：360
请输入还款方式（1为等额本金，2为等额本息）：2
请输入贷款年利率（直接回车为默认利率4.9%）：

第1个月需还:5307.27,其中本金1223.93,利息4083.33。本金余额:998776.07
第2个月需还:5307.27,其中本金1228.93,利息4078.34。本金余额:997547.13
第3个月需还:5307.27,其中本金1233.95,利息4073.32。本金余额:996313.18
第4个月需还:5307.27,其中本金1238.99,利息4068.28。本金余额:995074.20
第5个月需还:5307.27,其中本金1244.05,利息4063.22。本金余额:993830.15

.....

第356个月需还:5307.27,其中本金5200.23,利息107.04。本金余额:21014.11
第357个月需还:5307.27,其中本金5221.46,利息85.81。本金余额:15792.65
第358个月需还:5307.27,其中本金5242.78,利息64.49。本金余额:10549.87
第359个月需还:5307.27,其中本金5264.19,利息43.08。本金余额:5285.68
第360个月需还:5307.27,其中本金5285.68,利息21.58。本金余额:0.00

等额本息贷款支付的利息总额为：910616.19元

练习7： 可配置的问候语生成器

练习内容：

- 定义一个高级函数 `greeting_maker(prefix, suffix)`。这个函数返回一个匿名函数。
- 外部函数 `greeting_maker` 接收 `prefix`（前缀，如 "亲爱的"）和 `suffix`（后缀，如 "同学，你好！"）。
- 内部匿名函数接收一个 `name` 参数。
- 内部函数的功能是：将 `prefix`、`name`、`suffix` 拼接成一个完整的问候字符串并返回。

python

期望的执行效果

```
school_greeting = greeting_maker("尊敬的", "教授，您好！")
```

```
print(school_greeting("李华")) # 输出：  
尊敬的 李华 教授，您好！
```

```
party_greeting = greeting_maker("嘿，", "欢迎来到派对吧！🎉")
```

```
print(party_greeting("小明")) # 输出：  
嘿， 小明 欢迎来到派对吧！🎉
```

Happy coding! 🐍



第1题：学生成绩分析系统（函数+列表/字典+循环）

编写一个学生成绩管理系统，包含以下功能函数：

- `input_scores()`: 循环接收学生姓名和3门课的成绩，存储到字典中（键为姓名，值为成绩列表），直到输入空姓名为止。
- `calc_personal_avg(scores_dict)`: 计算每个学生的平均分，返回一个新字典（键为姓名，值为平均分）。
- `calc_class_avg(scores_dict)`: 计算全班每门课的平均分，返回一个列表，包含3门课的平均分。
- `get_top_student(scores_dict)`: 找出总分最高的学生，返回该学生的姓名和总分。

输入示例：

请输入学生姓名（回车结束）：张三
请输入语文成绩：85
请输入数学成绩：90
请输入英语成绩：78
请输入学生姓名（回车结束）：李四
请输入语文成绩：92
请输入数学成绩：88
请输入英语成绩：94
请输入学生姓名（回车结束）：

输出示例：

=== 成绩分析报告 ===

个人平均分：{'张三': 84.33, '李四': 91.33}

全班各科平均分：语文:88.5, 数学:89.0, 英语:86.0

第一名：李四，总分：274

第2题：通讯录管理系统（函数+字典+列表+集合）

编写一个通讯录管理系统，实现以下功能函数：

- `add_contact(contacts, name, phone)`：添加联系人到字典。
- `remove_contact(contacts, name)`：删除指定联系人。
- `search_contact(contacts, keyword)`：根据关键词（姓名或电话号码）搜索联系人，返回匹配的字典。
- `list_by_initial(contacts)`：将联系人按姓氏首字母分组，返回一个字典，键为首字母（大写），值为该首字母的所有联系人姓名列表。
- `find_duplicate_phones(contacts)`：找出重复的电话号码，返回一个集合。

测试数据：

```
contacts = {}  
add_contact(contacts, "张三", "13800001111")  
add_contact(contacts, "张伟", "13800002222")  
add_contact(contacts, "李四", "13800001111") # 电话重复  
add_contact(contacts, "王芳", "13800003333")  
add_contact(contacts, "赵六", "13800002222") # 电话重复
```

输出示例：

```
按首字母分组：{'Z': ['张三', '张伟'], 'L': ['李四'], 'W': ['王芳']}  
重复的电话号码：{'13800001111', '13800002222'}  
搜索"张"的结果：{'张三': '13800001111', '张伟': '13800002222'}  
}
```

第3题：火车票订购系统（函数+列表推导式+字典+集合运算）

某火车线路有以下座位（座位号按行号+列号命名，如"A1"表示第1排A座）：

- 座位布局：10排×4列（A、B、C、D）
- 已有订单记录现有的已售座位

实现以下函数：

- `generate_all_seats(rows, cols)`：使用列表推导式生成所有座位号列表。
- `get_available_seats(all_seats, booked_seats)`：使用集合运算获取可用座位。
- `book_seats(booked_seats, names, seats)`：批量订票，如果所有座位都可用则订票成功，否则提示哪些座位已被占用。
- `recommend_seats(available_seats, count)`：推荐连续的空座位（同一行连续），返回最靠前的连续座位组合。

测试数据：

```
all_seats = generate_all_seats(10, ['A','B','C','D'])
booked = {"A1", "A2", "B1", "C3", "C4", "D2", "D5", "E3", "F1"}
names = ["张三", "李四", "王五"]
seats_to_book = ["A3", "A4", "B2"] # 要订的座位
```

输出示例：

总座位数：40

已售座位：9

可用座位数：31

订票结果：成功！已为 张三,李四,王五 订票：['A3', 'A4', 'B2']

推荐2个连续座位：['A5', 'A6']

第4题：电影推荐系统（函数+字典+集合+列表排序）

设计一个简单的电影推荐系统，根据用户的观影历史，推荐相似用户喜欢的电影。

实现以下函数：

- `build_user_movies(records)`: 将原始观影记录（用户-电影列表）转换为字典，键为用户，值为该用户看过的电影集合。
- `calc_similarity(user1_movies, user2_movies)`: 计算两个用户的相似度（使用杰卡德相似系数 = 交集大小 / 并集大小）。
- `find_similar_users(user_movies, target_user, top_n=2)`: 找到与目标用户最相似的TOP N用户。
- `recommend_movies(user_movies, target_user, similar_users, top_k=3)`: 基于相似用户的观影历史，推荐目标用户没看过的电影，按推荐次数排序。

测试数据：

观影记录：元组列表（用户名，电影名）

```
records = [
    ("小明", "复仇者联盟"), ("小明", "泰坦尼克号"), ("小明", "阿凡达"),
    ("小红", "泰坦尼克号"), ("小红", "阿凡达"), ("小红", "盗梦空间"),
    ("小刚", "复仇者联盟"), ("小刚", "盗梦空间"), ("小刚", "星际穿越"),
    ("小丽", "阿凡达"), ("小丽", "泰坦尼克号"), ("小丽", "爱乐之城")
]
target = "小明"
```

输出示例：

用户观影记录：

小明: {'复仇者联盟', '泰坦尼克号', '阿凡达'}

小红: {'泰坦尼克号', '阿凡达', '盗梦空间'}

小刚: {'复仇者联盟', '星际穿越', '盗梦空间'}

小丽: {'阿凡达', '泰坦尼克号', '爱乐之城'}

与小明最相似的用户：小红(相似度0.67), 小刚(相似度0.33)

推荐给小明的电影：['盗梦空间', '星际穿越', '爱乐之城']

第5题：银行交易分析系统（挑战题）

设计一个银行交易分析系统，分析用户的收支记录，提供统计分析功能。

实现以下函数：

- `parse_transactions(lines)`: 解析原始交易记录（每行格式："日期, 类型, 金额, 分类"），返回交易对象列表。
- `summarize_by_category(transactions, trans_type)`: 按分类统计指定类型（收入/支出）的总金额，返回字典。
- `get_top_expenses(transactions, top_n=3)`: 获取金额最大的前N笔支出记录。
- `monthly_report(transactions)`: 生成月度报表，按月统计收入和支出，计算结余，返回列表（按月排序）。
- `detect_anomalies(transactions, threshold=2)`: 检测异常交易（支出超过平均支出threshold倍或收入超过平均收入threshold倍）。

测试数据：

```
raw_data = [  
    "2024-01-05,支出,120.50,餐饮",  
    "2024-01-10,收入,5000.00,工资",  
    "2024-01-15,支出,299.00,购物",  
    "2024-01-20,支出,45.00,交通",  
    "2024-02-01,收入,5000.00,工资",  
    "2024-02-10,支出,89.00,餐饮",  
    "2024-02-14,支出,1999.00,购物",  
    "2024-02-20,支出,60.00,交通",  
    "2024-02-28,支出,500.00,医疗",  
    "2024-03-05,收入,5000.00,工资",  
    "2024-03-10,支出,350.00,餐饮",  
    "2024-03-15,支出,1200.00,房租"  
]
```

输出示例：

=== 按分类统计支出 ===

餐饮: 559.5, 购物: 2298.0, 交通: 105.0, 医疗: 500.0, 房租: 1200.0

=== 支出最高的3笔 ===

1. 2024-02-14 | 购物 | 1999.0
2. 2024-03-15 | 房租 | 1200.0
3. 2024-02-28 | 医疗 | 500.0

=== 月度报表 ===

2024-01: 收入 5000.0, 支出 464.5, 结余 4535.5

2024-02: 收入 5000.0, 支出 2648.0, 结余 2352.0

2024-03: 收入 5000.0, 支出 1550.0, 结余 3450.0

=== 异常交易检测 ===

异常支出: ['2024-02-14 购物 1999.0', '2024-03-15 房租 1200.0']

异常收入: 无